

ME 595R

Deep Learning for Engineers

Syllabus

Schedule

Resources

© 2026. All rights reserved.

HW 9: Graph neural network with Newtonian Physics

We will reproduce a portion of [this paper](#). You will need to install [PyTorch Geometric](#). There is also a [code repository](#) and a demo that same link (the relevant files for you are models.py and the demo). You are welcome to refer to these, but should treat it like output from an AI chatbot (i.e., don't blindly copy/paste, but use to develop understanding). We are only doing a subset of the paper so don't get lost in the broader details of this code repo, mainly it will help you with some hyperparameters and GNN structure.

This homework focuses on the 2D spring case. We will not do the symbolic regression part (though I encourage you to read through that), and will just focus on the graph neural network training. I've pulled out data in the following [npz file](#) and started a script to process this data and provided additional guidance in the comments of the following [python script](#). In the

file I pulled out one set of training trajectories `train_traj` and one set of testing trajectories `train_traj`. The goal is to reasonably reproduce the testing trajectories with your graph neural net (the training trajectory is just provided as an additional guide since it should be a little easier to reproduce as its data will be in the training set). You will need to use [scipy.integrate.solve_ivp](#) to propagate the accelerations predicted by your GNN to compare the true and predicted trajectories (the time span ranges from 0 to 5). There is no need to animate or use different transparencies to indicate time. Just plot the full path like I do in my example. Make sure that you use the same colors for each particle in true vs. predicted, or put each particle on a separate plot, so we can clearly see the comparison between true and predicted.

Tips:

- In the demo they use a large message dimension (which corresponds to forces) and then add on an extra L1 loss function to encourage this dimension to be small. I just enforced the dimension to be small from the beginning (2 since that is what we know the dimension should be ideally). This is simpler and avoids the need for another loss function. Note that this works well for this scenario, but the opposite approach of allowing the network the extra freedom often allows better solutions (“L1” vs “bottleneck” in the paper).
- If you use a small message dimension like I did, you also won’t need your hidden dimension to be nearly so large.
- I only generated 1/10th of the data they did (and you may not even need all of that). I also did not use any data augmentation as they did in the paper. If we were doing symbolic regression then really tightening this up with more data would be justified, but for the purposes of this homework where we are just learning about GNNs the accuracy of our predictions will be plenty good.
- Note that they used MAE rather than MSE for their loss function.
- I didn’t need a learning rate scheduler, but YMMV.

